

ADLC

Orchestrare Agenti AI nello Sviluppo Software

Guida pratica al ciclo di vita AI-Driven per sviluppatori

Riccardo Rolla

2026

ADLC — Orchestrare Agenti AI nello Sviluppo Software

Guida pratica al ciclo di vita AI-Driven per sviluppatori

Copyright © 2026 Riccardo Rolla

Tutti i diritti riservati.

I nomi di prodotti, marchi registrati e aziende menzionati in questo libro sono proprietà dei rispettivi titolari. L'utilizzo in questo testo è esclusivamente a scopo identificativo e non implica alcuna affiliazione o approvazione.

Convenzioni tipografiche

Testo a spaziatura fissa — Comandi, nomi di file, codice sorgente

Grassetto — Concetti chiave, termini importanti

Corsivo — Nuovi termini, enfasi

Indice

Introduzione	ix
I Il Framework	1
1 ADLC: cos'è e perché esiste	3
1.1 I quattro problemi che ADLC risolve	3
1.2 Cosa NON è ADLC	4
1.3 Quando usarlo (e quando no)	5
1.4 I prerequisiti	5
1.5 I concetti fondamentali in sintesi	6
2 Setup del primo progetto in 15 minuti	7
2.1 Passo 1 — Copiare il framework nel repository	7
2.2 Passo 2 — Scaffold dello stato di progetto	8
2.3 Passo 3 — Compilare _CONTEXT.md	9
2.4 Passo 4 — Verificare con il validator	10
2.5 Passo 5 — Il primo turno con l'agente	10
2.6 Struttura raccomandata per .adlc/project/	11
3 Una sessione di lavoro reale	13
3.1 Inizio sessione — il bootstrap	13
3.2 Task LOW — una modifica senza freni	14
3.3 Task MEDIUM — proposta e approvazione	14
3.4 Task HIGH — l'HALT in azione	15
3.5 Il checkpoint	16
3.6 Fine sessione	16
3.7 Il ciclo completo in sintesi	17

II	La Memoria del Progetto	19
4	_CONTEXT.md: la fonte di verità	21
4.1	Dove vive e come viene trovato	21
4.2	La struttura del file	21
4.3	Sezioni opzionali utili	23
4.4	Come aggiornare _CONTEXT.md	23
4.5	Template minimo vs template completo	24
4.6	Errori comuni	24
5	PROGRESS.md e il cambio di fase	25
5.1	La differenza tra context e progress	25
5.2	Struttura di PROGRESS.md	25
5.3	Cosa scrivere in ogni voce	26
5.4	Come PROGRESS.md accelera le sessioni future	26
5.5	Il cambio di fase	27
5.6	Aggiornare PROGRESS.md senza sforzo	27
5.7	PROGRESS.md in team	27
6	Fasi e Modes	29
6.1	Le sette fasi del ciclo AI-Driven	29
6.2	I cinque Modes	29
6.3	Scenario tipico su TaskFlow API	30
6.4	Errori comuni con Fasi e Modes	31
III	Il Sistema di Sicurezza	33
7	Classificazione del rischio	35
7.1	I cinque livelli	35
7.2	Come l'agente classifica le richieste	37
7.3	Il risk floor per i Model Level	37

8	Model Levels: quale AI per quale task	39
8.1	La scala 1–7	39
8.2	Come stimare il livello di un task	40
8.3	I risk floor in dettaglio	40
9	Confidence Tag: FACT, INFERRED, ASSUMPTION	43
9.1	I tre tag	43
9.2	Quando i tag sono obbligatori	44
9.3	Il tag come segnale di azione	44
9.4	In pratica su TaskFlow API	45
10	HALT Triggers: il freno di emergenza	47
10.1	Il file halt-triggers.yaml	47
10.2	Override di progetto	48
10.3	I trigger non si disattivano mai con i Modes	48
11	Vincoli SEC e PERF	51
11.1	La libreria SEC (sicurezza)	51
11.2	La libreria PERF (performance)	51
11.3	Come attivare i vincoli in _CONTEXT.md	52
11.4	Come l’agente usa i vincoli	52
IV	Workflow Avanzato	55
12	Moduli e Skill: caricare solo ciò che serve	57
12.1	La distinzione tra moduli e skill	57
12.2	La mappa dei moduli	57
12.3	La mappa delle skill	57
12.4	Skill personalizzate di progetto	57
12.5	Il modulo 07_SPECIAL_LANES.md	59

13 Comandi conversazionali	61
13.1 La lista completa	61
13.2 I comandi più usati in dettaglio	61
13.3 Regole che i comandi non possono violare	63
13.4 Comandi personalizzati di progetto	63
14 Multi-agente: Claude, Copilot, Codex, Gemini	65
14.1 Uno stesso progetto, più entry point	65
14.2 Un workflow multi-agente su TaskFlow API	66
14.3 Mantenere Copilot allineato	66
14.4 I limiti del multi-agente	66
15 Monorepo, Company Extension e Codebase Legacy	69
15.1 Monorepo: un framework, più progetti	69
15.2 Company extension: processi SDLC aziendali	70
15.3 Codebase legacy: analisi e documentazione	70
V In Produzione	73
16 Troubleshooting, CI/CD e Manutenzione del Framework	75
16.1 Troubleshooting — problemi comuni	75
16.2 Integrazione CI/CD	76
16.3 Manutenzione del framework nel tempo	77
16.4 L'agente in Fase 6 — Ops	77
VI Appendici	79
A Glossario	81
B Comandi Conversazionali Completi	83
C Template Pronti all'Uso	85

D Risorse e Letture Consigliate	89
--	-----------

Introduzione

Hai già lavorato con un agente AI per qualche ora e hai visto come può essere straordinariamente utile. Gli chiedi di progettare un endpoint, lui risponde con codice pulito, test inclusi. Gli chiedi di rivedere un modulo, individua tre problemi che non avevi notato. Sembra quasi troppo bello.

Poi riapri il progetto il giorno dopo.

L'agente non ricorda nulla. Devi rispiegare lo stack tecnologico, i vincoli di sicurezza, il contesto del task. Riesci a farlo ricominciare, ma spendi venti minuti a ricostruire il contesto che ieri era già lì. La prossima settimana, un'altra mezz'ora. Nel giro di un mese, hai trascorso ore a *“riportare l'AI al punto in cui eravamo”*.

Questo è il problema senza nome. La maggior parte degli sviluppatori lo subisce, ci si abitua e lo mette in conto come “costo dell'AI”. Ma non deve essere così.

Tre storie vere

Storia 1 — La sessione dimenticata

Marco sta costruendo un'API per un'applicazione fintech. Ha discusso con Claude Code i requisiti di sicurezza: autenticazione OIDC, token JWT con rotazione, nessun segreto nei log. Ha passato un'ora a spiegare le scelte architetturali. L'agente ha prodotto codice eccellente.

Il giorno dopo, Marco riapre la sessione e chiede di aggiungere un endpoint. L'agente genera codice con `console.log(token)` per il debugging. Il vincolo sui log era stato discusso il giorno prima. L'agente non se lo ricorda.

Marco corregge manualmente. Non è un dramma, ma è evitabile.

Storia 2 — La modifica silenziosa

Giulia lavora con Copilot su uno schema PostgreSQL. Ha detto all'inizio che le migrazioni vanno sempre revisionate prima di essere eseguite — è una regola del team. Copilot lo sa, per ora.

Tre settimane dopo, nel bel mezzo di una funzionalità nuova, Copilot propone di aggiornare direttamente lo schema per “semplificare”. Giulia non se ne accorge subito. Lo schema viene modificato senza migration. Il deploy di venerdì diventa un problema.

La regola c'era. Nessuno la stava rispettando.

Storia 3 — La certezza inventata

Luca usa Codex per documentare un modulo legacy che nessuno tocca da tre anni. L'agente produce una documentazione ottima: architettura, dipendenze, flusso dati. Luca la distribuisce al team.

Due settimane dopo si scopre che tre delle funzioni documentate non esistono più. L'agente le aveva “inferite” dal contesto circostante, senza segnalare che stava facendo un'ipotesi. La documentazione era sbagliata dall'inizio.

Il problema comune

Queste tre storie hanno una radice comune: **gli agenti AI non hanno un sistema per mantenere il contesto tra sessioni, rispettare regole non scritte nel codice e segnalare l'incertezza in modo esplicito.**

Non è un problema degli agenti in sé — è strutturale. Un agente AI, per sua natura, inizia ogni sessione da zero a meno che non gli venga fornito un contesto esplicito. Se il contesto non è stato scritto da qualche parte in modo sistematico, va perso.

La soluzione non è trovare un agente “migliore”. La soluzione è costruire un sistema che sopravvive tra una sessione e l'altra.

Cos'è ADLC

ADLC — *AI-Driven Software Development Life Cycle* — è quel sistema. Non è un'AI, non è un tool da installare, non è un framework specifico per un linguaggio. È un insieme di file Markdown e JSON che vivono nel tuo repository e che dicono all'agente:

- dove sei nel progetto (fase, task attivo)
- cosa deve rispettare (vincoli di sicurezza e performance)
- quando deve fermarsi e chiedere (operazioni ad alto rischio)
- come classificare il proprio output quando non è certo

Il risultato è un agente che lavora *in continuità*: ogni sessione parte dal punto in cui la precedente si è fermata. Non perché l'agente “ricordi” — ma perché il contesto è scritto, strutturato e sempre disponibile.

Come leggere questo libro

Questo libro è diviso in cinque parti.

Parte I — Il Framework (Capitoli 1–3) ti porta da zero a una prima sessione di lavoro funzionante. Se vuoi testare ADLC subito, puoi leggere solo questa parte e tornare al resto quando ne hai bisogno.

Parte II — La Memoria del Progetto (Capitoli 4–6) spiega in dettaglio `_CONTEXT.md` e `PROGRESS.md`, i due file che costruiscono la continuità tra sessioni.

Parte III — Il Sistema di Sicurezza (Capitoli 7–11) copre la classificazione del rischio, i livelli di modello, i confidence tag, gli HALT trigger e i vincoli SEC/PERF.

Parte IV — Workflow Avanzato (Capitoli 12–15) affronta scenari più complessi: multi-agente, monorepo, codebase legacy, integrazione aziendale.

Parte V — In Produzione (Capitolo 16) chiude il ciclo con CI/CD, troubleshooting e manutenzione del framework nel tempo.

Il progetto di esempio

Per tutta la durata del libro useremo un progetto realistico: **TaskFlow API**, una REST API per la gestione di task e progetti, costruita con Node.js e Fastify. Il progetto è volutamente familiare — chiunque abbia già costruito una REST API riconoscerà i pattern — ma abbastanza ricco da far emergere i problemi reali che ADLC risolve.

Il team è composto da tre sviluppatori: **Lorenzo**, tech lead con esperienza backend, che usa Claude Code in terminale; **Giulia**, sviluppatrice full-stack che lavora in VS Code con GitHub Copilot; e **Marco**, che si occupa di DevOps e code review, e usa Claude Code e Codex CLI. Li seguiremo in sessioni diverse nel corso del libro.

Seguiremo TaskFlow API dall’analisi dei requisiti fino al deploy, passando per design, implementazione, test e release. A ogni capitolo vedremo come ADLC cambia il modo in cui l’agente AI lavora su quel progetto.

Non importa se il tuo stack è diverso. ADLC è agnostico rispetto alla tecnologia. Ciò che impari su TaskFlow API si applica identico a un progetto Python, Go, Flutter o qualsiasi altra combinazione.

Cominciamo.

Parte I

II Framework

1

ADLC: cos'è e perché esiste

TaskFlow API nasce da una decisione semplice: il team di tre sviluppatori vuole usare agenti AI per accelerare lo sviluppo, ma senza rinunciare alla qualità e senza perdere il controllo su ciò che viene generato.

Il primo giorno, Lorenzo — il tech lead — apre Claude Code e chiede di impostare il progetto. L'agente risponde subito con una struttura di directory, un `package.json` e i primi endpoint. Funziona. Il team è soddisfatto.

Il terzo giorno, però, Lorenzo nota che il codice generato nel pomeriggio non rispetta la convenzione di gestione degli errori che avevano discusso la mattina. L'agente ha dimenticato.

Il decimo giorno, il team ha tre conversazioni diverse con tre agenti diversi, ognuna con il proprio contesto implicito. Nessuno dei tre agenti sa cosa stanno facendo gli altri.

È a questo punto che Lorenzo decide di adottare ADLC.

1.1 I quattro problemi che ADLC risolve

ADLC nasce per risolvere quattro problemi specifici, tutti connessi alla natura degli agenti AI attuali.

Problema 1 — L'AI dimentica

Gli agenti AI non hanno memoria persistente tra sessioni. Ogni conversazione inizia da zero. Se hai discusso la settimana scorsa che il tuo stack usa OIDC per l'autenticazione e che i token non devono mai apparire nei log, dovrai ridirlo la settimana prossima.

Soluzione ADLC: `_CONTEXT.md`, un file Markdown alla radice del progetto che contiene tutto il contesto rilevante. L'agente lo legge a ogni sessione. Se cambia qualcosa, lo aggiorni. Non devi ripetere nulla.

Problema 2 — L'AI ignora i vincoli

Anche quando spieghi i requisiti di sicurezza all'inizio di una conversazione, l'agente può ignorarli o dimenticarli nel corso del lavoro — specialmente in sessioni lunghe o complesse. Un vincolo che non è strutturato in modo esplicito non viene rispettato sistematicamente.

Soluzione ADLC: I vincoli SEC-XX (sicurezza) e PERF-XX (performance) sono definiti una volta in `_CONTEXT.md` e riletti dall'agente prima di ogni operazione critica. Non sono una nota a margine: sono parte del protocollo obbligatorio.

Problema 3 — L'AI inventa

Gli agenti AI sono progettati per rispondere, non per dire “non lo so”. Questo produce output che mischiano fatti verificabili, deduzioni logiche e ipotesi non verificate — il tutto con la stessa sicurezza apparente.

Soluzione ADLC: I confidence tag (FACT / INFERRED / ASSUMPTION) obbligano l'agente a classificare esplicitamente la certezza del proprio output su decisioni ad alto impatto. Un ASSUMPTION non è un errore — è un segnale che devi verificare prima di procedere.

Problema 4 — L'AI fa troppo

Un agente senza limiti può modificare lo schema del database mentre aggiusta un bug di UI, o cambiare la configurazione di produzione mentre aggiorna un test. Le operazioni rischiose vengono eseguite senza avvisare.

Soluzione ADLC: Gli HALT trigger definiscono i path e le operazioni che richiedono conferma esplicita prima di procedere. L'agente si ferma, spiega cosa sta per fare e aspetta il tuo via libera.

1.2 Cosa NON è ADLC

Prima di andare avanti, è importante chiarire cosa ADLC non è.

Non è un'AI. ADLC non genera codice, non esegue comandi, non decide nulla in autonomia. È un sistema di file che struttura il dialogo tra te e l'agente AI che già usi.

Non è un tool da installare. Non c'è un pacchetto npm, una pip install, un binario. Il framework è un insieme di file Markdown e JSON che vivono nel tuo repository. Non c'è un server, non c'è una dashboard, non ci sono dipendenze esterne.

Non è specifico per un linguaggio o stack. ADLC funziona identico su un progetto Node.js, Python, Go, Rust, Flutter. Il contenuto dei file cambia (i tuoi vincoli specifici, il tuo stack), ma il sistema è lo stesso.

Non è un sostituto del processo di sviluppo. ADLC si inserisce nel processo che già hai — o aiuta a costruirlo, se non ce l’hai. Non è un framework Agile alternativo, non è una metodologia di project management.

1.3 Quando usarlo (e quando no)

ADLC è uno strumento per progetti reali. Ha senso adottarlo quando:

- Stai lavorando su un progetto che dura più di qualche giorno.
- Usi agenti AI regolarmente, non solo per domande occasionali.
- Il progetto ha vincoli di sicurezza, performance o compliance che devono essere rispettati sistematicamente.
- Vuoi usare più agenti diversi (Claude, Copilot, Codex) sullo stesso progetto senza duplicare le regole.
- Lavori in team e vuoi che tutti gli agenti seguano le stesse convenzioni.

Non serve quando:

- Stai facendo un esperimento di un’ora o un prototipo usa-e-getta.
- Hai solo una domanda veloce da fare all’agente.
- Il progetto è talmente semplice che non ha vincoli significativi.

SUGGERIMENTO

Costo di adozione: circa 15 minuti per lo scaffold iniziale, poi 1–2 minuti a sessione per mantenere `_CONTEXT.md` aggiornato. Il tempo si recupera entro la prima settimana di lavoro continuativo.

1.4 I prerequisiti

Per usare ADLC non servono competenze avanzate, ma tre basi sono necessarie.

Riga di comando. Devi saper aprire un terminale (PowerShell o Bash) e lanciare comandi. I tool del framework (`init.ps1`, `validate.ps1`) sono script da eseguire, non interfacce grafiche.

Markdown base. I file del framework sono Markdown: titoli, tabelle, liste. Devi saper aprire, leggere e modificare un `.md`. Non serve sapere tutto il Markdown — bastano i costrutti di base.

Git base. `clone`, `commit`, `push`, `branching` di base. Tecnicamente puoi usare ADLC senza Git su un progetto locale, ma è fortemente sconsigliato — il versionamento del contesto è parte del valore.

Utili ma non obbligatori: YAML (per gli HALT trigger), JSON (per il manifest), PowerShell scripting (per modificare i tool). Capirai questi file leggendoli; non è necessario scriverli da zero.

1.5 I concetti fondamentali in sintesi

Prima di entrare nel dettaglio nei capitoli successivi, ecco una mappa dei concetti principali del framework.

Concetto	File/Meccanismo	A cosa serve
Stato del progetto	_CONTEXT.md	Fase, task, stack, vincoli
Storia del progetto	PROGRESS.md	Registro sessioni e decisioni
Fasi	Phase: in _CONTEXT.md	Ciclo 0–6
Modes	Mode: in _CONTEXT.md	Livello di cerimonia
Rischio	Classificazione interna	LOW / MEDIUM / HIGH / CRITICAL
Model Level	Campo nel task	Potenza AI necessaria (1–7)
Confidence tag	FACT / INFERRED / ASSUMPTION	Certezza dell'output
HALT Trigger	halt-triggers.yaml	Path con conferma obbligatoria
Vincoli SEC	SEC-XX in _CONTEXT.md	Sicurezza riusabile
Vincoli PERF	PERF-XX in _CONTEXT.md	Performance riusabile
Moduli	.adlc/modules/	Regole per fase
Skill	.adlc/modules/skills/	Guide tematiche

Tabella 1.1: Concetti fondamentali del framework ADLC

Riepilogo

- ADLC risolve quattro problemi: la dimenticanza, l'ignoranza dei vincoli, l'invenzione e l'eccesso di autonomia degli agenti AI.
- Non è un tool, non è un'AI, non è specifico per uno stack tecnologico.
- Funziona tramite file Markdown e JSON che vivono nel repository e danno all'agente un contesto strutturato e persistente.
- Ha senso su progetti reali con continuità tra sessioni; non serve per esperimenti brevi.
- Costo di adozione: ~15 minuti iniziali, poi 1–2 minuti per sessione.

Nel prossimo capitolo installiamo ADLC su TaskFlow API.

2

Setup del primo progetto in 15 minuti

Lorenzo ha clonato il repository di TaskFlow API, aperto il terminale e ha davanti a sé una directory con il codice del progetto e nient'altro. Nessun `_CONTEXT.md`, nessun framework AI. Questo è il punto di partenza.

In questo capitolo seguiamo Lorenzo — e tu con lui — mentre aggiunge ADLC al progetto, compila il contesto per la prima volta e fa il primo turno con un agente AI. Quindici minuti, non di più.

2.1 Passo 1 — Copiare il framework nel repository

ADLC non ha un installer. Si copia. La struttura che ti serve è questa:

Listing 2.1: Struttura del framework nel repository

```
progetto/
+-- AGENTS.md      ← OpenAI Codex CLI
+-- CLAUDE.md      ← Claude Code
+-- GEMINI.md      ← Google Gemini
+-- OPENCLAW.md    ← OpenClaw
+-- .github/
|   +-- copilot-instructions.md ← GitHub Copilot
+-- .adlc/
    +-- halt-triggers.yaml
    +-- manifest.json
    +-- modules/    ← regole del framework
    +-- schemas/    ← schemi JSON
    +-- tools/      ← script di utilità
```

i NOTA

L'URL del repository è indicativo — verifica quello aggiornato nella documentazione ufficiale del framework o nel file `README.md` della distribuzione che hai ricevuto.

Clona il framework in una directory temporanea e copia i file:

Listing 2.2: Installazione su Bash (Linux/macOS/WSL)

```
git clone https://github.com/rollaradio/adlc-framework /tmp/adlc
cp /tmp/adlc/AGENTS.md .
cp /tmp/adlc/CLAUDE.md .
cp -r /tmp/adlc/.github .
cp -r /tmp/adlc/.adlc .
```

Listing 2.3: Installazione su PowerShell (Windows)

```
git clone https://github.com/rollaradio/adlc-framework $env:TEMP\adlc
Copy-Item "$env:TEMP\adlc\AGENTS.md" .
Copy-Item "$env:TEMP\adlc\CLAUDE.md" .
Copy-Item "$env:TEMP\adlc\.github" . -Recurse
Copy-Item "$env:TEMP\adlc\.adlc" . -Recurse
```

i NOTA

I file `AGENTS.md`, `CLAUDE.md`, ecc. sono read-only per l'uso normale — non modificarli. Le tue personalizzazioni vanno in `.adlc/project/` (lo vediamo al §2.6).

2.2 Passo 2 — Scaffold dello stato di progetto

Esegui lo script di inizializzazione per creare i file di stato:

Listing 2.4: Inizializzazione su Bash

```
bash .adlc/tools/init.sh
```

Listing 2.5: Inizializzazione su PowerShell

```
.\.adlc\tools\init.ps1
```

Lo script crea quattro cose:

Per progetti piccoli o spike usa il context minimo:

File/Cartella	A cosa serve
_CONTEXT.md	Stato corrente (fase, task, vincoli)
PROGRESS.md	Diario di bordo delle sessioni
.adlc/project/instructions.md	Regole specifiche per il progetto
.adlc/project/skills/	Skill personalizzate

Tabella 2.1: File creati dallo script di inizializzazione

Listing 2.6: Inizializzazione con context minimale

```
bash .adlc/tools/init.sh --context minimal
```

2.3 Passo 3 — Compilare _CONTEXT.md

Apri _CONTEXT.md e compila i campi con i valori del tuo progetto. Ecco quello di Lorenzo per TaskFlow API:

Listing 2.7: _CONTEXT.md di TaskFlow API — setup iniziale

```
| Param          | Value          |
|-----|-----|
| Project        | TaskFlow API  |
| Phase         | 0-Discovery    |
| Mode          | STANDARD       |
| Active Task    | T-001 Setup e analisi requisiti |
| Active Task Token Est. | 8000/2000/10000 |
| Active Task Model Level | 3 Sonnet Low  |

## Stack
| Layer | Technology |
|-----|-----|
| Runtime | Node.js 22 |
| Framework | Fastify 5 |
| Database | PostgreSQL 16 + Prisma |
| Auth | OIDC + JWT |

## Security Constraints
| SEC-01 | Input Validation | Zod schema su ogni endpoint |
| SEC-02 | Authentication | OIDC, JWT con refresh rotation |
| SEC-03 | Logging | Nessun token o PII nei log |

## Performance Constraints
| PERF-01 | Latency | P95 < 200ms per endpoint |
| PERF-02 | Payload size | Max 1MB per risposta |
```

SUGGERIMENTO

Compila almeno Project, Phase, Mode e Stack. I vincoli SEC e PERF sono fondamentali — anche solo due o tre, quelli più rilevanti. Il resto puoi aggiungerlo progressivamente.

2.4 Passo 4 — Verificare con il validator

Listing 2.8: Validazione su Bash

```
bash .adlc/tools/validate.sh
```

Listing 2.9: Validazione su PowerShell

```
.\.adlc\tools\validate.ps1
```

Output atteso:

Listing 2.10: Output del validator

```
ADLC validation - v3.3.0

[OK] Framework files present
[OK] AGENTS.md, CLAUDE.md, GEMINI.md, OPENCLAW.md found
[OK] halt-triggers.yaml valid
[OK] manifest.json valid
[!] _CONTEXT.md: Active Task Token Est. is placeholder (0/0/0)

ADLC validation passed (2 warnings)
```

I warning sui placeholder sono normali per la prima sessione. Con `--strict` diventano failure (utile in CI).

2.5 Passo 5 — Il primo turno con l'agente

Apri l'agente AI e chiedi la conferma del contesto:

“Carica `_CONTEXT.md` e confermami il contesto: fase, task attivo, stack, vincoli SEC e PERF attivi.”

L'agente risponde con qualcosa del tipo:

Listing 2.11: Risposta di bootstrap dell'agente

```
Context: TaskFlow API | Phase: 0-Discovery | Task: T-001 |
Stack: Node.js/Fastify/PostgreSQL |
Constraints: SEC-01,SEC-02,SEC-03,PERF-01. Proceed?
```

Quando vedi questa conferma, sai che l'agente ha letto il contesto. Da questo momento puoi iniziare il dialogo reale senza ripetere nulla.

2.6 Struttura raccomandata per .adlc/project/

Listing 2.12: Struttura della cartella di personalizzazione

```
.adlc/project/
+-- instructions.md  ← regole specifiche del progetto
+-- domain.md       ← business logic e regole di dominio
+-- conventions.md   ← convenzioni di codice
+-- skills/          ← skill personalizzate
```

Per TaskFlow API, Lorenzo aggiunge a instructions.md:

Listing 2.13: .adlc/project/instructions.md di TaskFlow API

```
# TaskFlow API - Project Instructions

## Conventions
- All API errors must use RFC 7807 Problem Details format
- Database queries use Prisma only - no raw SQL except in migrations
- All endpoints must have a corresponding integration test

## Git workflow
- Branch naming: feat/T-NNN-description, fix/T-NNN-description
- PR review required before merge to main
- Squash merge only
```

Riepilogo

1. Copia i file del framework nella radice del progetto
2. Esegui `init.ps1` / `init.sh` per lo scaffold iniziale
3. Compila `_CONTEXT.md` con fase, stack e vincoli
4. Verifica con `validate.ps1` / `validate.sh`
5. Fai il primo bootstrap con l'agente e ricevi conferma del contesto

TaskFlow API è pronta. L'agente sa dove sei, cosa stai costruendo e cosa non può ignorare. Nel prossimo capitolo vediamo come si svolge una sessione completa, dall'inizio al commit finale.

3

Una sessione di lavoro reale

È martedì mattina. Lorenzo apre il terminale, avvia Claude Code e inizia la sessione su TaskFlow API. Seguiamo ogni passaggio, dal bootstrap al commit finale, per vedere ADLC in azione su un caso concreto.

i NOTA

TaskFlow API ha attraversato le fasi 0–2 nelle settimane precedenti. I requisiti sono stati analizzati, l’architettura definita (schema DB, contratto API con 12 endpoint), le decisioni aperte chiuse. Lorenzo ha aggiornato `_CONTEXT.md` a Phase: 3-Implementation al termine del design. Da questo punto inizia il lavoro che seguiamo ora.

Il task di oggi: implementare l’endpoint `POST /tasks` — il cuore dell’API, quello che crea un nuovo task per un utente autenticato.

3.1 Inizio sessione — il bootstrap

Lorenzo non spiega nulla all’agente. Chiede solo:

“Carica `_CONTEXT.md` e confermami il contesto.”

L’agente risponde:

Listing 3.1: Conferma di bootstrap

```
Context: TaskFlow API | Phase: 3-Implementation |
Task: T-007 POST /tasks endpoint |
Stack: Node.js 22/Fastify 5/PostgreSQL 16 |
Constraints: SEC-01,SEC-02,SEC-03,PERF-01,PERF-02. Proceed?
```

In tre secondi, l'agente sa tutto. Lorenzo scrive: *“Procedi.”*

SUGGERIMENTO

Pattern da seguire: all'inizio di ogni sessione, chiedi sempre la conferma del contesto prima di qualsiasi istruzione. È il modo per verificare che l'agente abbia letto `_CONTEXT.md` correttamente.

3.2 Task LOW — una modifica senza freni

Lorenzo comincia con un task leggero: aggiungere JSDoc all'interfaccia Task già esistente. È una modifica LOW-risk: documentazione, nessun impatto sulla logica.

“Aggiungi JSDoc all'interfaccia Task in `src/types/task.ts`. Segui lo stile già usato in `src/types/user.ts`.”

L'agente produce il JSDoc coerente con lo stile del progetto e notifica:

Listing 3.2: Notifica per task LOW

```
[LOW] Added JSDoc to Task interface (4 fields documented).  
No risk items. Changes in: src/types/task.ts
```

Per i task LOW, l'agente esegue e notifica senza aspettare approvazione.

3.3 Task MEDIUM — proposta e approvazione

Lorenzo chiede di implementare l'endpoint principale:

“Implementa `POST /tasks`. L'endpoint riceve titolo, descrizione e scadenza, crea il task per l'utente autenticato, e risponde 201 con il task creato.”

L'agente classifica come MEDIUM (nuova feature) e risponde con un piano prima di scrivere codice:

Listing 3.3: Piano proposto per task MEDIUM

```
[MEDIUM] Piano per POST /tasks:  
  
1. Route handler in src/routes/tasks.ts  
   - Validazione: Zod schema (SEC-01)
```

```

- userId dal JWT, non dall'input (SEC-02)
- Nessun userId nei log (SEC-03)

2. Service in src/services/taskService.ts
- createTask(userId, data) → Prisma
- Errori: 400 / 401 / 500 in formato RFC 7807

3. Test in tests/routes/tasks.test.ts
- 201 dati validi, 400 titolo mancante,
- 401 no auth, 400 titolo troppo lungo

Stima: ~80 righe codice + ~60 test. Model Level: 3.
Procedo?

```

Lorenzo approva. L'agente implementa citando SEC-02 e SEC-03 nel codice generato — i vincoli sono attivi, non devono essere ripetuti.

3.4 Task HIGH — l'HALT in azione

Lorenzo nota che lo schema Prisma non ha un indice su userId nella tabella tasks. Chiede all'agente di aggiungerlo.

L'agente si ferma:

Listing 3.4: HALT per modifica schema

```

[!] HALT - trigger: schema (HIGH)

File: prisma/schema.prisma
Pattern: **/prisma/schema.prisma

Proposta: aggiunta @@index([userId]) in model Task.
Richiede migration:
  prisma migrate dev --name add-task-userid-index

Rischio: MEDIUM - migration non-destructive.
In produzione: lock temporaneo sulla tabella.

Minimum model level: 5. Conferma esplicita richiesta.
Procedo?

```

Lorenzo risponde: *“Confermato. Procedi con schema e migration.”*

L'agente aggiunge l'indice e genera il file di migration. Il processo di HALT ha reso espliciti i dettagli che meritavano attenzione.

3.5 Il checkpoint

Dopo circa un'ora di lavoro, Lorenzo manda un checkpoint:

```
@checkpoint
```

L'agente risponde:

Listing 3.5: Output di @checkpoint

```
Checkpoint - TaskFlow API | 2026-06-27 10:47

Completato:
- POST /tasks: route, service, Zod, RFC 7807, 4 test
- Migration: add-task-userid-index

In corso: niente di sospeso. Blocchi: nessuno.

Aggiornamento _CONTEXT.md consigliato:
  Active Task → T-007.2 GET /tasks
  Active Task Token Est. → 5000/1500/6500

Applico gli aggiornamenti?
```

Lorenzo dice sì. L'agente produce il blocco da incollare in `_CONTEXT.md`.

3.6 Fine sessione

Lorenzo chiude la sessione con un checkpoint esplicito:

```
@checkpoint — chiudo la sessione.
```

L'agente produce:

- Il blocco aggiornato per `_CONTEXT.md` (Active Task, Token Est.)
- Una voce per `PROGRESS.md` con riepilogo del lavoro

Lorenzo copia i due blocchi, fa il commit, chiude il terminale. Domani l'agente saprà esattamente dove erano rimasti.

3.7 Il ciclo completo in sintesi

Listing 3.6: Schema del ciclo ADLC per sessione

```
INIZIO SESSIONE
+- Chiedi conferma contesto → agente legge _CONTEXT.md

DURANTE LA SESSIONE
+- Task LOW    → esegue + notifica
+- Task MEDIUM → propone piano → approvi → esegue
+- Task HIGH   → HALT → spiega rischi → confermi → esegue
+- @checkpoint → ogni 3-5 azioni significative

FINE SESSIONE
+- @checkpoint → aggiorna _CONTEXT.md + PROGRESS.md → commit
```

Riepilogo

- Il **bootstrap** assicura che l'agente abbia letto `_CONTEXT.md` prima di qualsiasi lavoro.
- I task **LOW** vengono eseguiti direttamente; i **MEDIUM** richiedono approvazione del piano; i **HIGH** richiedono conferma esplicita dopo un **HALT**.
- Il **checkpoint** ogni 3–5 azioni fissa un punto fermo e mantiene i file di stato aggiornati.
- La **fine sessione** è un checkpoint esplicito che produce gli aggiornamenti da committare.

Nei prossimi capitoli esploriamo in dettaglio i meccanismi che rendono possibile tutto questo, a partire dal cuore del sistema: `_CONTEXT.md`.

Parte II

La Memoria del Progetto

4

_CONTEXT.md: la fonte di verità

In un progetto software tradizionale, il contesto vive nelle teste delle persone. Con gli agenti AI il problema è amplificato: ogni sessione è un “ingresso di qualcuno di nuovo”. L’agente non ha memoria. Senza un documento strutturato, sei tu che ricostruisci il contesto a voce ogni volta.

_CONTEXT.md è la risposta a questo problema. È la fonte di verità del progetto: un singolo file Markdown che l’agente legge all’inizio di ogni sessione e che contiene tutto ciò che serve per lavorare correttamente.

4.1 Dove vive e come viene trovato

_CONTEXT.md vive alla radice del tuo progetto. L’agente segue questo protocollo di ricerca:

1. Cerca _CONTEXT.md nella directory corrente.
2. Se non lo trova, risale nelle directory padre fino alla radice del repository.
3. Se ne trova più di uno (monorepo), usa quello più vicino al file su cui sta lavorando.
4. Se non trova nulla, chiede di crearne uno.

4.2 La struttura del file

Un _CONTEXT.md completo ha quattro sezioni principali.

Sezione 1 — Context card (obbligatoria)

Listing 4.1: Context card di TaskFlow API

Param	Value	
-------	-------	--

-----		-----
Project		TaskFlow API
Phase		3-Implementation
Mode		STANDARD
Active Task		T-007.2 GET /tasks con filtri
Active Task Token Est.		5000/1500/6500
Active Task Model Level		3 Sonnet Low
Active Branch		feat/T-007-tasks-endpoints
Last Checkpoint		2026-06-27 10:47 - POST /tasks completato

Phase (0–6) determina quali moduli il framework carica. **Mode** controlla il livello di cerimonia (Capitolo 6). **Active Task Token Est.** è la stima input/output/totale in token. **Active Task Model Level** (1–7) è il livello AI necessario.

Sezione 2 — Stack (obbligatoria)

Listing 4.2: Stack tecnologico nel context

## Stack		
Layer		Technology

Runtime		Node.js 22
Framework		Fastify 5
Database		PostgreSQL 16 + Prisma ORM
Auth		OIDC + JWT (jose library)
Testing		Vitest + Supertest
Deploy		Docker + Railway

L’agente usa questa tabella per fare scelte coerenti senza dover chiedere.

Sezione 3 — Vincoli di sicurezza (obbligatoria se applicabile)

Listing 4.3: Vincoli SEC nel context

## Security Constraints		
ID		Name
-----		Spec

SEC-01		Input Validation
SEC-02		Authentication
SEC-03		Logging

		Zod schema obbligatorio	
		OIDC, JWT con rotazione	
		Nessun token o PII nei log	

Sezione 4 — Vincoli di performance (obbligatoria se applicabile)

Listing 4.4: Vincoli PERF nel context

```
## Performance Constraints
| ID      | Name          | Target          |
|-----|-----|-----|
| PERF-01 | Latency       | P95 < 200ms    |
| PERF-02 | Payload Size  | Max 1MB per risposta|
```

4.3 Sezioni opzionali utili

Decisioni architetturali aperte

Listing 4.5: Open Decisions nel context

```
## Open Decisions
| ID      | Decision          | Options          | Deadline |
|-----|-----|-----|-----|
| AD-01   | Paginazione cursor/offset | cursor / offset | 2026-07-05 |
```

Se una decisione è qui, l'agente non la prende da solo — ti chiede di decidere.

Note per l'agente

Listing 4.6: Note nel context

```
## Agent Notes
- Priorità: qualità > velocità
- Test: integration test su DB reale, no mock di Prisma
- Naming: snake_case DB, camelCase TS, kebab-case file
```

4.4 Come aggiornare _CONTEXT.md

Aggiorna il file ogni volta che cambia:

- La fase del progetto
- Il task attivo
- I vincoli SEC o PERF
- Una decisione aperta viene presa

Il modo più pratico è il comando `@context-update` (Capitolo 13).

4.5 Template minimo vs template completo

Il framework offre due template in `.adlc/modules/templates/`:

- **CONTEXT_TEMPLATE.md** — completo, con tutte le sezioni incluse decisioni aperte e note.
- **CONTEXT_MIN.md** — minimo per spike e POC: solo context card, stack e vincoli essenziali.

SUGGERIMENTO

Inizia sempre dal minimo. Un `_CONTEXT.md` parzialmente compilato è meglio di uno vuoto. Aggiungi sezioni quando ne senti la necessità.

4.6 Errori comuni

Lasciare i placeholder. Il validator segnala warning se Active Task Token Est. contiene `0/0/0`. Non blocca il lavoro, ma indica contesto incompleto.

Non aggiornare il file. Un `_CONTEXT.md` non aggiornato è un contesto che mente. Dopo una settimana senza aggiornamenti, l'agente lavora su informazioni obsolete.

Mettere troppo nel context. Le regole dettagliate del progetto vanno in `.adlc/project/instructions.md`. Il context contiene lo stato attuale, non il manuale del progetto.

Non committare il file. `_CONTEXT.md` va in git. È parte del progetto, non un file temporaneo.

Riepilogo

- `_CONTEXT.md` è il singolo file che dà all'agente il contesto strutturato per lavorare correttamente senza istruzioni ripetute.
- Contiene: context card, stack, vincoli SEC e PERF, e sezioni opzionali.
- Va aggiornato a ogni cambio di fase, task o vincolo.
- Inizia dal template minimo; aggiungi sezioni man mano che il progetto cresce.
- Committalo in git e non lasciarlo mai stale.

5

PROGRESS.md e il cambio di fase

_CONTEXT.md fotografa il presente: dove sei adesso, cosa stai facendo, quali vincoli rispettare. Ma non risponde a domande come “perché abbiamo scelto Prisma invece di Drizzle?” o “cosa è successo nella sessione di giovedì scorso quando il deploy è andato storto?”

Quelle risposte vivono in PROGRESS.md.

5.1 La differenza tra context e progress

_CONTEXT.md	PROGRESS.md
Stato attuale	Storia del progetto
Si sovrascrive	Cresce solo in avanti
Letto sempre dall'agente	Letto su richiesta
Fotografa dove sei	Registra da dove sei venuto
Conciso per design	Può essere lungo

Tabella 5.1: Differenza tra _CONTEXT.md e PROGRESS.md

Pensa a _CONTEXT.md come al cruscotto di un'auto — mostra la velocità attuale, il carburante, la temperatura. Pensa a PROGRESS.md come al libretto di manutenzione — registra ogni intervento, ogni anomalia, ogni decisione presa nel tempo.

5.2 Struttura di PROGRESS.md

Il file cresce dall'alto verso il basso: le voci più recenti sono in cima.

Listing 5.1: Esempio di PROGRESS.md per TaskFlow API

```
# PROGRESS – TaskFlow API

---

## 2026-07-03 – T-009 Auth middleware refactor

- Estratto il middleware JWT in src/middleware/auth.ts
- Identificato: Zod eseguiva dopo auth – corretto l'ordine

DECISIONE: middleware condiviso invece di per-route.
Motivazione: riduce duplicazione, comportamento coerente.

---

## 2026-06-27 – T-007 POST /tasks (70%)

- POST /tasks: Zod, auth JWT, RFC 7807, 4 test passanti
- Migration: add-task-userid-index (dev only)
- HALT trigger su schema.prisma → confermato e proceduto
```

5.3 Cosa scrivere in ogni voce

Una voce utile contiene tre tipi di informazioni:

Fatto — cosa è stato prodotto o modificato concretamente.

Scoperta — qualcosa di non ovvio trovato durante il lavoro: bug latenti, comportamenti inaspettati delle librerie, assunzioni sbagliate.

Decisione — una scelta architetturale, di design o di processo, con la motivazione.

Le scoperte e le decisioni sono le più preziose. Sono le cose che non si deducono dal codice — sono il contesto dietro al codice.

5.4 Come PROGRESS.md accelera le sessioni future

Giulia — non presente giovedì durante il refactor — apre una sessione e chiede:

“Carica _CONTEXT.md e PROGRESS.md, poi dimmi cosa è cambiato nell’autenticazione nelle ultime due settimane.”

L’agente risponde in trenta secondi con un riepilogo completo del refactor, la decisione architetturale presa e il lavoro ancora da fare. Giulia ha il contesto senza aver bisogno di leggere la git history o chiedere a Lorenzo.

5.5 Il cambio di fase

Una delle voci più importanti è il cambio di fase:

Listing 5.2: Voce di cambio fase in PROGRESS.md

```
## 2026-07-15 - CAMBIO FASE: Design → Implementation

Stack confermato: Node.js 22 / Fastify 5 / PostgreSQL 16
Decisioni architetturali prese:
- Paginazione: cursor-based (AD-01 chiusa)
- Cache: nessuna al primo rilascio (AD-02 rimandata)

API contract: 12 endpoint (vedi docs/api-contract.md)
Schema DB v1 migrato in dev

Primo task Phase 3: T-001 Setup struttura e dipendenze
```

Questa voce serve da spartiacque: tutto sopra è storia di design, tutto sotto è storia di implementazione.

5.6 Aggiornare PROGRESS.md senza sforzo

Alla fine di ogni sessione usa @checkpoint:

@checkpoint — proponi una voce per PROGRESS.md

L'agente produce una voce pronta. Tu la revisioni, aggiungi il contesto che solo tu conosci, e la incolli nel file. Trenta secondi di lavoro.

5.7 PROGRESS.md in team

In un team di tre persone, PROGRESS.md diventa il canale di comunicazione asincrona sul progetto. Ogni sviluppatore scrive la propria voce alla fine della sessione.

ATTENZIONE

PROGRESS.md non sostituisce la comunicazione di team. Non è una board kanban, non è un sistema di ticket. È un diario tecnico che integra il contesto degli agenti AI. Tenete separati i due strumenti.

Riepilogo

- PROGRESS.md registra la storia del progetto: fatto, scoperte, decisioni con motivazione.
- Si aggiorna a fine sessione con @checkpoint; tu revisioni e incolli.
- Le **scoperte** e le **decisioni** (con motivazione) sono le voci più preziose.
- Il cambio di fase è una voce speciale che documenta lo spartiacque tra una fase e l'altra.
- In team, permette agli agenti di ricostruire il contesto delle sessioni di altri membri.

6

Fasi e Modes

Uno degli errori più comuni nell'uso degli agenti AI è trattarli come strumenti uniformi: la stessa configurazione per qualsiasi task, in qualsiasi momento del progetto.

Le **Fasi** dicono all'agente in che punto del ciclo di sviluppo si trova. I **Modes** controllano quanta cerimonia applicare per quella sessione. Insieme, rendono l'agente contestuale invece che generico.

6.1 Le sette fasi del ciclo AI-Driven

Fase	Nome	Quando ci sei
0	Discovery	Raccolta requisiti, comprensione del problema
1	Analysis	Analisi requisiti, scelta dell'approccio macro
2	Design	Architettura, API contract, schema DB
3	Implementation	Codice, test
4	Verification	QA, integration testing
5	Release	Deploy, documentazione finale
6	Ops	Monitoraggio, incident response, manutenzione

Tabella 6.1: Le sette fasi del ciclo ADLC

Il campo Phase in `_CONTEXT.md` indica la fase corrente e determina quale modulo il framework carica:

6.2 I cinque Modes

LITE — lavoro quotidiano

Per lavoro routine su un progetto stabile. Riduce l'overhead senza rinunciare alla sicurezza.

Fase	Modulo	Skill suggerite
0–1	02_DISCOVERY_ANALYSIS.md	SKILL_ANALYSIS.md
2	03_DESIGN.md	SKILL_DESIGN.md, SKILL_API_DESIGN.md
3	04_IMPLEMENTATION.md	SKILL_API_DESIGN.md, SKILL_SECURITY.md
4–5	05_VERIFICATION_RELEASE.md	SKILL_TESTING.md
6	06_OPS.md	SKILL_OPS.md

Tabella 6.2: Moduli e skill per fase

- Conferma di sessione: saltata se `_CONTEXT.md` non è cambiato
- Confidence tag: solo per output >10 righe o classificati HIGH
- Checkpoint: suggerito ogni 5+ azioni (non obbligatorio per LOW/MEDIUM)
- HALT trigger: sempre rispettati

STANDARD — default

Applica il protocollo completo. Per nuovi progetti, nuovi team, zone del codice mai toccate.

AUDIT — tracciabilità completa

Per lavoro con compliance, sicurezza, revisioni formali. Confidence tag su ogni output, checklist di completamento per ogni task.

RAPID — spike/emergenze

Lavoro time-boxed (1–2 ore), nessuna documentazione obbligatoria. Usa sempre un branch separato; evita i path di produzione. SEC critici sempre rispettati.

FAST — task semplici

Output diretto senza loop di conferma, a meno che il rischio non sia HIGH.

6.3 Scenario tipico su TaskFlow API

Per cambiare mode aggiorna il campo Mode: in `_CONTEXT.md`. Esempio concreto:

Lorenzo, dopo due settimane in STANDARD, passa a LITE:

Periodo	Mode	Motivazione
Fase 0–2	STANDARD	Decisioni architetturali ad alto impatto
Prima settimana Fase 3	STANDARD	Nuovo territorio, stabilire convenzioni
Settimane 2–N Fase 3	LITE	Pattern rodati, team allineato
Audit pre-release	AUDIT	Tracciabilità completa richiesta
Hotfix urgente	RAPID	Velocità, branch dedicato
Refactor di naming	FAST	Task a basso rischio

Tabella 6.3: Evoluzione del Mode durante il progetto

Listing 6.1: Aggiornamento del Mode nel context

```
| Mode | LITE |
```

Alla sessione successiva, l’agente non chiede la conferma di bootstrap — la presuppone invariata. Lorenzo scrive “*Implementa DELETE /tasks/:id*” e l’agente propone subito il piano senza il passaggio di conferma contesto. Il flusso è più fluido, le regole di sicurezza sono le stesse.

6.4 Errori comuni con Fasi e Modes

Tenere STANDARD per tutta la vita del progetto. Dopo le prime settimane, le conferme diventano rumore. Il team inizia a ignorarle. Abbassa a LITE quando il progetto è stabile.

Usare RAPID per lavoro che va in produzione. RAPID è per esperimenti. Se il codice prodotto in RAPID finisce in main senza review, stai usando il mode sbagliato.

Non aggiornare la fase in `_CONTEXT.md` quando si cambia fase. L’agente carica i moduli sbagliati. Il cambio di fase nel `PROGRESS.md` deve essere accompagnato dall’aggiornamento del campo `Phase` nel `context`.

Pensare che i Modes disattivino gli HALT trigger. Non è così. In nessun mode gli HALT trigger vengono disattivati.

Riepilogo

- Le **sette fasi** (0–6) definiscono il punto del ciclo di sviluppo e quale modulo viene caricato.
- I **cinque Modes** (LITE, STANDARD, AUDIT, RAPID, FAST) controllano quanta cerimonia applica l’agente.
- Mode e Fase sono indipendenti: puoi essere in Fase 3 con Mode LITE.
- Gli HALT trigger non vengono mai disattivati, indipendentemente dal Mode.

- Cambia Mode liberamente durante il progetto: STANDARD per partire, LITE per il lavoro quotidiano rodato.

Parte III

Il Sistema di Sicurezza

7

Classificazione del rischio

Ogni richiesta che fai a un agente AI porta con sé un rischio implicito. Rinominare una variabile è diverso da modificare lo schema del database. ADLC formalizza questa distinzione in cinque livelli di rischio.

7.1 I cinque livelli

LOW

Modifiche a basso impatto, facilmente reversibili, che non toccano logica di business o sistemi condivisi. Esempi: rinomina variabile, typo, JSDoc, formattazione.

Cosa fa l'agente: esegue direttamente e notifica.

Listing 7.1: Notifica LOW

```
[LOW] Fixed typo in error message in src/services/taskService.ts.  
No risk items.
```

MEDIUM

Nuove funzionalità, refactor locali, modifiche multi-file reversibili con git revert. Esempi: nuovo endpoint, estrazione funzione, nuovo set di test.

Cosa fa l'agente: propone un piano dettagliato, aspetta la tua approvazione.

Listing 7.2: Proposta MEDIUM

```
[MEDIUM] Piano per GET /tasks con filtri:  
1. Route handler con query params (status, assignee, cursor)
```

```
2. Zod schema per validazione params (SEC-01)
3. Query Prisma con where dinamico + paginazione cursor
Procedo?
```

HIGH

Modifiche difficilmente reversibili o con effetti su sistemi condivisi. Esempi: schema DB, autenticazione, architettura, eliminazione dati, configurazioni deploy.

Cosa fa l'agente: HALT, piano dettagliato con analisi rischi, attende conferma esplicita.

Listing 7.3: HALT HIGH

```
[!] HALT - HIGH risk detected

Operazione: aggiunta colonna NOT NULL completed_at a tabella tasks

Rischi:
- Migration non-reversibile (richiede backfill)
- Lock sulla tabella durante la migration su PostgreSQL

Piano: (1) colonna nullable → (2) backfill → (3) NOT NULL
Conferma esplicita richiesta. Procedi?
```

HIGH+

Operazioni su secrets, credenziali, compliance, dati PII. L'agente non propone nemmeno un piano finché non riceve una conferma di intento.

Listing 7.4: HALT HIGH+

```
[X] HALT - HIGH+ risk: auth/secrets path detected

File: src/middleware/auth.ts
Pattern: **/middleware/auth*

Conferma che vuoi modificare il codice di autenticazione.
```

CRITICAL

Decisioni mission-critical ambigue con implicazioni architetturali irreversibili. L'agente presenta alternative e un ADR bozza. **Non esegue nulla.**

Listing 7.5: HALT CRITICAL

[X] HALT – CRITICAL: architectural decision required

Questione: strategia paginazione GET /tasks (AD-01 aperta)

- Cursor-based: performante, no jump a pagina N
- Offset-based: jump supportato, degrada con tabelle grandi

Raccomandazione: cursor-based (FACT – PERF-01: P95 < 200ms)

ma la scelta dipende da requisiti UI non ancora definiti.

Non procedo finché AD-01 è aperta.

7.2 Come l'agente classifica le richieste

L'agente esamina tre fattori prima di agire:

1. Il path del file corrisponde a un pattern negli HALT trigger?
2. L'operazione è additiva (basso rischio) o distruttiva/strutturale?
3. La modifica è locale (un file) o sistemica (autenticazione, schema DB)?

7.3 Il risk floor per i Model Level

La classificazione del rischio influenza anche il modello AI minimo richiesto:

Livello	Minimum Model Level
LOW	1
MEDIUM	3
HIGH	5
HIGH+ (auth, secrets, compliance, produzione)	6
CRITICAL	7

Tabella 7.1: Risk floor per Model Level

Riepilogo

- Cinque livelli: LOW (esegui), MEDIUM (piano → approva → esegui), HIGH (HALT → conferma), HIGH+ (HALT → conferma intento → conferma esecuzione), CRITICAL (HALT → alternative → nessuna esecuzione).

- La classificazione considera path HALT, tipo di operazione e ambito dell'effetto.
- Ogni livello ha un risk floor che forza un minimum model level.
- La classificazione è sempre comunicata prima di agire — puoi correggerla.

8

Model Levels: quale AI per quale task

C'è un errore comune nell'uso degli agenti AI: usare sempre il modello più potente disponibile. Il primo motivo è il costo: i modelli più potenti costano di più in token. Il secondo è la latency: su task semplici e frequenti la differenza è percepibile e non giustificata.

ADLC introduce i Model Levels per risolvere questo: una scala da 1 a 7 che mappa la complessità del task al modello appropriato.

8.1 La scala 1–7

Level	Token range (totali)	A cosa serve
1	< 4k	Piccoli edit, docs, formattazione
2	4k–8k	Modifiche localizzate, test semplici
3	8k–16k	Implementazione standard, debugging
4	16k–32k	Feature multi-file, design moderato
5	32k–64k	Refactor complessi, sicurezza
6	64k–120k	Architettura, debugging profondo
7	> 120k	Mission-critical, alta ambiguità

Tabella 8.1: Scala dei Model Level ADLC

Il mapping ai modelli concreti (Anthropic, OpenAI, Gemini) vive in `.adlc/manifest.json#model_levels` e si legge con:

Listing 8.1: Visualizzazione mapping modelli

```
bash .adlc/tools/show-models.sh # Linux/macOS/WSL
.\.adlc\tools\show-models.ps1 # Windows PowerShell
```

8.2 Come stimare il livello di un task

Passo 1 — Stima dei token

La stima è input / output / totale. Alcune euristiche pratiche:

Input	Token approssimativi
File da 100 righe	500–800
_CONTEXT.md compilato	500–1.000
3 file da 200 righe	3.000–5.000
Modulo da 10 file	10.000–20.000

Tabella 8.2: Euristiche per la stima dei token in input

Esempio per TaskFlow API — Lorenzo implementa GET /tasks con filtri:

- Input: _CONTEXT.md (700) + route esistente (600) + service (400) + schema (300) = ~ 2.000
- Output: nuovo endpoint + test ~ 2.500
- Totale: ~ 4.500 token $\rightarrow$ range 4k–8k $\rightarrow$ **Livello 2** (dalla tabella 8.1)

Passo 2 — Applica il risk floor

Il task è classificato MEDIUM. Il risk floor per MEDIUM è livello 3. Anche se la stima token suggerisce livello 2, il level sale a 3.

Listing 8.2: AI Sizing in _CONTEXT.md

```
| Active Task Token Est. | 2000/2500/4500 |
| Active Task Model Level | 3 Sonnet Low |
```

8.3 I risk floor in dettaglio

Il risk floor è cumulativo: un task HIGH (floor 5) che tocca auth (floor 6) usa il massimo — livello 6.

Riepilogo

- I Model Level (1–7) mappano la complessità del task al modello AI appropriato.

Condizione	Minimum Level
Task MEDIUM	3
Task HIGH	5
Auth, secrets, compliance, produzione	6
Architettura o cambiamenti cross-modulo	6
CRITICAL	7

Tabella 8.3: Risk floor per condizione

- La stima è input/output/totale in token; il mapping vendor concreto è in `manifest.json`.
- Il risk floor forza un minimum level indipendentemente dalla stima token.
- La stima migliora con la pratica; le prime settimane è normale sbagliare.

9

Confidence Tag: FACT, INFERRED, ASSUMPTION

Gli agenti AI hanno un problema strutturale: rispondono con la stessa sicurezza apparente sia quando sanno, sia quando deducono, sia quando inventano. I confidence tag sono il meccanismo con cui ADLC rende esplicita questa distinzione.

9.1 I tre tag

I tag seguono sempre lo stesso formato, in coda all'affermazione che classificano:

Listing 9.1: Formato dei confidence tag

```
AI CONFIDENCE: [FACT | INFERRED | ASSUMPTION]
Basis: [motivazione]
[TODO: azione suggerita – obbligatorio per ASSUMPTION su HIGH-risk]
```

FACT

L'agente ha verificato l'informazione leggendo direttamente il codice, un file, o un output di test presente nel contesto.

Listing 9.2: Esempio FACT

```
La funzione getUserById lancia UserNotFoundError su record mancante.

AI CONFIDENCE: FACT
Basis: letto src/services/userService.ts, riga 45-50
```

INFERRED

L'agente ha dedotto l'informazione da contesto indiretto: nome della funzione, struttura del codice, convenzioni del framework. La deduzione è logica ma non verificata direttamente.

Listing 9.3: Esempio INFERRED

```
Il middleware usa @fastify/jwt basandosi sull'import in src/app.ts.  
  
AI CONFIDENCE: INFERRED  
Basis: import @fastify/jwt in src/app.ts; non ho letto la  
configurazione completa del middleware
```

ASSUMPTION

L'agente ha fatto un'ipotesi non supportata da evidenza diretta nel contesto.

Listing 9.4: Esempio ASSUMPTION

```
Ho assunto che tasks abbia un indice su user_id (PERF-01: P95 < 200ms).  
  
AI CONFIDENCE: ASSUMPTION  
Basis: non ho letto lo schema Prisma  
TODO: verifica index su prisma/schema.prisma prima di andare in prod
```

9.2 Quando i tag sono obbligatori

Sempre obbligatori:

- Output classificati HIGH-risk o superiore
- Claim su vincoli SEC-XX o PERF-XX
- Codice che modifica path presenti negli HALT trigger
- Affermazioni su comportamento del codice esistente senza averlo letto

Consigliati ma non obbligatori in Mode LITE:

- Output MEDIUM superiori a 10 righe
- Deduzioni su comportamento di librerie esterne

9.3 Il tag come segnale di azione

Tag	Azione
FACT	Usa l'informazione. Verifica solo se la fonte potrebbe essere obsoleta.
INFERRED	Verifica prima di agire su operazioni MEDIUM o superiori.
ASSUMPTION	Stop. Verifica prima di qualsiasi operazione.

Tabella 9.1: Azione da intraprendere per ogni confidence tag

9.4 In pratica su TaskFlow API

Lorenzo chiede all'agente di rivedere la gestione errori di `POST /tasks`. L'agente risponde con tre claim di certezza diversa:

- **FACT**: la validazione Zod è corretta — verificato leggendo il file.
- **INFERRED**: il middleware è probabilmente registrato nell'ordine giusto — consigliato di verificare.
- **ASSUMPTION**: il mapping degli errori Prisma potrebbe non esistere — Lorenzo deve verificare prima di andare in staging.

Senza i confidence tag, Lorenzo avrebbe ricevuto lo stesso testo senza sapere cosa è stato verificato e cosa no.

Riepilogo

- Tre tag: FACT (verificato), INFERRED (dedotto), ASSUMPTION (ipotesi).
- FACT: usa. INFERRED: verifica prima di operazioni MEDIUM+. ASSUMPTION: stop, verifica subito.
- Obbligatorie su output HIGH-risk, claim SEC/PERF, codice in zone HALT.
- In Mode LITE si riducono ai soli casi critici.

10

HALT Triggers: il freno di emergenza

Ci sono zone in ogni progetto che non dovresti mai modificare di impulso: schema del database, codice di autenticazione, credenziali, CI/CD. Gli HALT trigger sono il meccanismo con cui ADLC presidia queste zone.

10.1 Il file halt-triggers.yaml

Vive in `.adlc/halt-triggers.yaml` ed è già lì dopo init. I sei trigger predefiniti:

Listing 10.1: Trigger predefiniti in halt-triggers.yaml

```
triggers:
- id: schema
  patterns: ["**/migrations/**", "**/prisma/schema.prisma",
            "**/*.sql", "**/alembic/**"]
  risk: HIGH

- id: auth
  patterns: ["**/auth/**", "**/middleware/auth*",
            "**/oauth/**"]
  risk: HIGH+

- id: secrets
  patterns: ["**/.env*", "**/secrets/**", "**/*.pem"]
  risk: HIGH+

- id: infra
  patterns: ["**/terraform/**", "**/k8s/**",
            "**/Dockerfile*", "**/*.tf"]
  risk: HIGH

- id: cicd
```

```

patterns: [".github/workflows/**", "**/.gitlab-ci.yml"]
risk: HIGH

- id: framework
  patterns: [".adlc/modules/**", "AGENTS.md", "CLAUDE.md"]
  risk: HIGH

```

10.2 Override di progetto

Le zone sensibili specifiche del tuo progetto vanno in `.adlc/project/halt-triggers.yaml`:

Listing 10.2: Override di progetto per TaskFlow API

```

version: 1

triggers:
  - id: billing
    patterns: [ "**/billing/**", "**/payments/**" ]
    reason: Logica di pagamento - impatto finanziario
    risk: HIGH+

```

Il file di progetto *aggiunge* trigger a quelli predefiniti. Per disattivare un trigger predefinito temporaneamente:

Listing 10.3: Disattivazione temporanea del trigger schema

```

version: 1
disable:
  - schema # disattivato durante la fase di design intensivo DB

```

ATTENZIONE

Un trigger disattivato è una zona senza rete di sicurezza. Documenta in `PROGRESS.md` perché l'hai disattivato e quando pianifichi di riattivarlo.

10.3 I trigger non si disattivano mai con i Modes

Anche in Mode FAST — il mode con meno cerimonia — un path che corrisponde a un trigger viene bloccato e richiede conferma. Il rischio di un percorso auth o di uno schema DB non cambia in base a quanto velocemente vuoi lavorare quel giorno.

Riepilogo

- I sei trigger predefiniti coprono: schema DB, auth, secrets, infra, CI/CD, framework ADLC.
- Gli override di progetto in `.adlc/project/halt-triggers.yaml` estendono (o disattivano) i trigger predefiniti.
- I trigger non si disattivano mai con i Modes — sono il livello di sicurezza costante.
- Un falso positivo (trigger attivato per path poi non modificato) è accettabile e preferibile al contrario.

11

Vincoli SEC e PERF

Ogni progetto software ha requisiti di sicurezza e performance. Il problema è che questi requisiti vengono discussi una volta — e poi evaporano dalla pratica quotidiana. I vincoli SEC e PERF di ADLC risolvono questo: sono una libreria di requisiti riusabili, definiti una volta nel framework, attivati in `_CONTEXT.md`, riletti dall'agente prima di ogni operazione critica.

11.1 La libreria SEC (sicurezza)

Nove vincoli predefiniti in `SEC_CONSTRAINTS.md`:

ID	Nome	Principio
SEC-01	Input Validation	Allow-list, validazione lato server obbligatoria
SEC-02	Authentication	Provider collaudato, token short-lived, no log secrets
SEC-03	Authorization	Least privilege, validazione ownership
SEC-04	Secrets & Config	No hardcoded secrets, env vars, rotazione chiavi
SEC-05	Data Protection	PII minimizzata, redaction log, TLS
SEC-06	Dependency Hygiene	Pin versioni, SBOM, vulnerability scan
SEC-07	Threat Modeling	Asset, attori, STRIDE, rischio residuo (Fase 0–2)
SEC-08	Logging & Monitoring	Log eventi sicurezza, strutturati JSON, no PII
SEC-09	SSRF	Allow-list URL in uscita, blocco IP privati

Tabella 11.1: Libreria vincoli SEC

11.2 La libreria PERF (performance)

Sette vincoli predefiniti in `PERF_CONSTRAINTS.md`:

ID	Nome	Principio
PERF-01	Latency Targets	SLO per endpoint (P50/P95/P99)
PERF-02	Throughput & Concurrency	RPS target, async I/O, no blocking
PERF-03	Database Efficiency	Indici, no N+1, bound result sizes
PERF-04	Resource Utilization	Limiti CPU/memoria, profiling
PERF-05	Caching & CDN	Cache layer, TTL, invalidazione
PERF-06	Startup & Warm-up	Lazy loading, health check readiness
PERF-07	Build & Bundle Size	Code splitting, tree shaking (frontend)

Tabella 11.2: Libreria vincoli PERF

11.3 Come attivare i vincoli in `_CONTEXT.md`

Non tutti i vincoli si applicano a ogni progetto. Attivi solo quelli rilevanti specificando la configurazione specifica del tuo contesto:

Listing 11.1: Vincoli attivati per TaskFlow API

```
## Security Constraints
| SEC-01 | Input Validation | Zod schema obbligatorio |
| SEC-02 | Authentication | OIDC + JWT short-lived, refresh rotation |
| SEC-03 | Authorization | task.userId === auth.userId |
| SEC-05 | Data Protection | Pino redaction: password, token, email |

## Performance Constraints
| PERF-01 | Latency | P95 < 200ms – monitored via Grafana |
| PERF-03 | DB Efficiency | Max 3 query per request, take: 50 |
```

La colonna Spec è la parte più importante: non “usa autenticazione” — scrivi la specifica concreta che l’agente deve rispettare.

11.4 Come l’agente usa i vincoli

L’agente rilegge i vincoli attivi in due momenti:

- Prima di generare codice in zone SEC-sensibili (autenticazione, input utente, logging, chiamate esterne)
- Prima di piani HIGH o superiori (il piano deve citare esplicitamente i vincoli rilevanti)

Il comando `@show-constraints` mostra in qualsiasi momento i vincoli attivi nella sessione corrente.

Riepilogo

- SEC-01..09 coprono: input, autenticazione, autorizzazione, secrets, dati, dipendenze, threat modeling, logging, SSRF.
- PERF-01..07 coprono: latency, throughput, database, risorse, cache, startup, bundle.
- Si attivano in `_CONTEXT.md` con la specifica concreta — non il principio generico.
- L'agente li rilegge prima di codice SEC-sensibile e li cita nei piani HIGH+.
- Vincoli personalizzati si aggiungono con ID arbitrari (es. SEC-10, PERF-08).

Parte IV

Workflow Avanzato

12

Moduli e Skill: caricare solo ciò che serve

Il framework ADLC è composto da file che non vengono caricati tutti insieme all’inizio di ogni sessione. Vengono caricati selettivamente, in base alla fase e al tipo di task. Questo evita il “context overload” — la situazione in cui l’agente ha così tante regole da non riuscire ad applicarne nessuna bene.

12.1 La distinzione tra moduli e skill

I moduli (.adlc/modules/NN_*.md) definiscono le regole operative del framework per ciascuna fase. Sono le istruzioni strutturali: come comportarsi, cosa verificare, quando fermarsi. Sono read-only.

Le skill (.adlc/modules/skills/SKILL_*.md) sono guide tematiche specializzate. Definiscono come affrontare un tipo specifico di lavoro.

12.2 La mappa dei moduli

12.3 La mappa delle skill

12.4 Skill personalizzate di progetto

Le skill del framework sono generiche. Il tuo progetto può avere esigenze specifiche in .adlc/project/skills/:

Listing 12.1: Skill di dominio per TaskFlow API

```
# SKILL - TaskFlow Domain
```

File	Fase	Quando si carica
00_MODE.md	Sempre	Regole per ogni Mode
01_CORE_RULES.md	Sempre	Regole base, risk, halt, confidence tag
02_DISCOVERY_ANALYSIS.md	0–1	Discovery e analisi requisiti
03_DESIGN.md	2	Architettura e design
04_IMPLEMENTATION.md	3	Implementazione e test
05_VERIFICATION_RELEASE.md	4–5	QA e release
06_OPS.md	6	Operazioni e monitoring
07_SPECIAL_LANES.md	Su richiesta	Hotfix, spike, parallel track
08_PROMPT_LIBRARY.md	Su richiesta	Template di prompt
09_CODEBASE_ANALYSIS.md	Su richiesta	Analisi codebase esistente
10_DOCUMENTATION.md	Su richiesta	Generazione documentazione
11_BUGFIX_PLAYBOOK.md	Su richiesta	Debug strutturato
SEC_CONSTRAINTS.md	Su richiesta	Libreria vincoli SEC
PERF_CONSTRAINTS.md	Su richiesta	Libreria vincoli PERF

Tabella 12.1: Mappa dei moduli ADLC

File	Tipo di lavoro
SKILL_ANALYSIS.md	Analisi requisiti, scenari d'uso
SKILL_DESIGN.md	Architettura, pattern, decisioni strutturali
SKILL_API_DESIGN.md	API REST: naming, status code, paginazione
SKILL_DATA_ACCESS.md	Schema DB, ORM, query, migration
SKILL_SECURITY.md	Revisione sicurezza, SEC-XX in pratica
SKILL_TESTING.md	Strategie di test, piramide, mocking
SKILL_UI.md	Componenti frontend, accessibilità
SKILL_OPS.md	CI/CD, monitoring, incident response

Tabella 12.2: Skill disponibili nel framework

```
> Carica quando: lavori su task, progetti, assegnazioni.

## Regole di Dominio
- Un Task appartiene esattamente a un Project
- Transizioni stato: draft → open → in_progress → done | cancelled
- done e cancelled sono stati terminali
- completed_at viene impostato automaticamente su status → done

## Business Invariants
- Un utente non può essere assegnato a un task a cui non ha accesso
- Cancellazioni: soft delete (isDeleted = true, non physical delete)
```

12.5 Il modulo 07_SPECIAL_LANES.md

Copre tre casi fuori dal ciclo normale:

Hotfix lane: bug critico in produzione. Branch da main, fix minimale, review obbligatoria, rollback plan documentato prima del deploy.

Spike lane: esplorazione time-boxed (1–2 ore). Branch dedicato, obiettivo dichiarato, risultato in `PROGRESS.md` anche se lo spike fallisce.

Parallel track: feature su più layer (API + frontend). Tre sync point obbligatori: accordo sul contratto API, integrazione, E2E.

ATTENZIONE

Non modificare mai `.adlc/modules/`. Le personalizzazioni vanno in `.adlc/project/` — questo garantisce che il framework possa essere aggiornato senza perdere le tue regole.

Riepilogo

- I **moduli** sono le regole per ogni fase — caricati automaticamente in base a Phase.
- Le **skill** sono guide tematiche — caricate su richiesta in base al tipo di task.
- Skill personalizzate di progetto in `.adlc/project/skills/` hanno priorità sulle skill del framework.
- I moduli del framework sono read-only: personalizza in `.adlc/project/`.

13

Comandi conversazionali

I comandi conversazionali sono istruzioni che dai all'agente nella chat, precedute dal prefisso @. Non sono comandi shell. Sono segnali semantici che l'agente riconosce e interpreta secondo il protocollo ADLC.

13.1 La lista completa

Comando	Scopo	Modifica file?
@checkpoint	Fissa stato avanzamento	Propone (no auto)
@context-update	Aggiorna il context	Propone (no auto)
@show-constraints	Mostra vincoli SEC/PERF	No
@security-check	Verifica vs SEC attivi	No
@perf-check	Verifica vs PERF attivi	No
@load-phase [N]	Carica modulo fase N	No
@set-phase [N]	Propone cambio fase	Propone (no auto)
@alternatives	Genera 2–3 opzioni	No (ADR per CRITICAL)
@simplify	Riduce scope	Propone (no auto)
@rollback	Piano di rollback	Propone (no auto)
@stop	Ferma immediatamente	No

Tabella 13.1: Comandi conversazionali ADLC

13.2 I comandi più usati in dettaglio

@checkpoint

Il più usato. Ogni 3–5 azioni significative o prima di una pausa.

Listing 13.1: Output tipico di @checkpoint

```
Checkpoint – TaskFlow API | 2026-06-27 14:32

Completato:
- POST /tasks: route, service, Zod, RFC 7807, 4 test
- GET /tasks: filtri, paginazione cursor, 6 test

Aggiornamento _CONTEXT.md consigliato:
  Active Task → T-008 GET /tasks/:id
  Active Task Token Est. → 3000/1000/4000
Applico gli aggiornamenti?
```

Importante: _CONTEXT.md e PROGRESS.md non vengono mai modificati automaticamente. L'agente propone il testo; sei tu ad applicarlo.

@security-check

Revisione del codice/piano contro i vincoli SEC attivi. Ogni claim include il confidence tag.

Listing 13.2: Output di @security-check su POST /tasks

```
SEC-01 Input Validation: [OK]
  Zod schema con titolo obbligatorio, dueDate ISO 8601. [FACT]

SEC-02 Authentication: [OK]
  JWT verificato dal middleware auth. [FACT]

SEC-05 Data Protection: [!] DA VERIFICARE
  Non ho verificato che il log non esponga il body completo.
  [ASSUMPTION – TODO: verifica src/app.ts configurazione Pino]

Azione richiesta: verifica SEC-05 prima del merge.
```

@alternatives

Genera opzioni strutturate con pro, contro e raccomandazione.

@simplify

Quando il task diventa troppo grande per una sessione: produce uno split in sotto-task con motivazione.

13.3 Regole che i comandi non possono violare

1. Nessun comando bypassa gli HALT trigger.
2. Nessun comando modifica file automaticamente.
3. @security-check cita solo i vincoli in _CONTEXT.md.
4. @rollback non esegue azioni distruttive senza conferma.
5. I comandi non cambiano la classificazione del rischio.

13.4 Comandi personalizzati di progetto

In `.adlc/project/instructions.md`:

Listing 13.3: Comandi personalizzati per TaskFlow API

```
## Project Commands

@deploy-check
  Esegui in sequenza:
  1. @security-check sull'endpoint modificato
  2. Verifica che tutti i test passino
  3. Controlla backward-compatibility della migration
  4. Proponi la voce di CHANGELOG.md
```

Riepilogo

- I comandi @ sono segnali semantici — non comandi shell.
- I più usati: @checkpoint, @context-update, @security-check, @alternatives, @simplify, @stop.
- Non bypassano il protocollo di sicurezza; non modificano file automaticamente.
- Comandi personalizzati in `.adlc/project/instructions.md` trasformano workflow ricorrenti in shortcut.

14

Multi-agente: Claude, Copilot, Codex, Gemini

ADLC è progettato per funzionare con più agenti, simultaneamente o in sequenza, sullo stesso progetto, con lo stesso `_CONTEXT.md` come fonte di verità condivisa.

14.1 Uno stesso progetto, più entry point

Agente	File di startup	Note
Claude Code	CLAUDE.md	Importa AGENTS.md
OpenAI Codex CLI	AGENTS.md	Entry point principale
Google Gemini	GEMINI.md	Delega ad AGENTS.md
OpenClaw	OPENCLAW.md	Delega ad AGENTS.md
GitHub Copilot	.github/copilot-instructions.md	Non può importare

Tabella 14.1: Entry point per ogni agente AI

La struttura è a stella: `AGENTS.md` è il centro, gli altri file orbitano intorno. Il contenuto sostanziale del protocollo — le regole di rischio, i confidence tag, gli `HALT` trigger — vive in `AGENTS.md`.

SUGGERIMENTO

Regola d'oro: modifica solo `AGENTS.md` per le regole comuni. Non modificare `CLAUDE.md`, `GEMINI.md`, `OPENCLAW.md` per aggiungere regole — aggiornali solo per comportamenti specifici di quell'agente.

14.2 Un workflow multi-agente su TaskFlow API

- **Lorenzo** usa Claude Code per design e architettura
- **Giulia** usa GitHub Copilot in VS Code per lo sviluppo quotidiano
- **Marco** usa Claude Code per code review, Codex CLI per script

Il `_CONTEXT.md` è condiviso in git. Ogni mattina, quando Lorenzo fa il bootstrap, Claude Code legge lo stesso file che Giulia ha aggiornato la sera prima. Il contesto è allineato senza sincronizzazione manuale.

14.3 Mantenere Copilot allineato

Copilot non può importare file esterni, quindi il suo file di istruzioni deve essere mantenuto allineato con `AGENTS.md`:

Listing 14.1: Verifica allineamento Copilot

```
bash .adlc/tools/sync-copilot.sh
```

Listing 14.2: Output di sync-copilot

```
[OK] Risk classification: present in both
[OK] HALT trigger reference: present in both
[!] @alternatives command: present in AGENTS.md,
    missing in copilot-instructions.md
```

Esegui `sync-copilot` dopo ogni modifica a `AGENTS.md` e prima di aggiungere un nuovo membro al team che usa Copilot.

14.4 I limiti del multi-agente

Non è collaborazione in tempo reale. Gli agenti non si parlano tra loro. La coerenza è garantita dal `_CONTEXT.md` condiviso, non da un canale diretto. Conflitti git su edit concorrenti rimangono conflitti git.

La qualità del context condiviso determina la qualità del risultato. Se `_CONTEXT.md` è obsoleto, tutti gli agenti lavorano su informazioni sbagliate.

Riepilogo

- `AGENTS.md` è il centro del sistema multi-agente; gli altri file sono wrapper.

- Il `_CONTEXT.md` condiviso in git garantisce la coerenza tra agenti diversi.
- `sync-copilot` verifica l'allineamento di Copilot dopo ogni modifica alle regole comuni.
- Il multi-agente funziona bene per lavoro asincrono; non risolve i conflitti di edit concorrenti.

15

Monorepo, Company Extension e Codebase Legacy

I capitoli precedenti hanno seguito TaskFlow API come progetto singolo. Questo capitolo copre tre scenari avanzati che emergono con la complessità reale del lavoro di squadra.

15.1 Monorepo: un framework, più progetti

In un monorepo, ogni sottoprogetto che ha bisogno di un contesto indipendente crea il proprio `_CONTEXT.md`:

Listing 15.1: Struttura di un monorepo ADLC

```
taskflow/  
+-- apps/  
|   +-- api/          ← REST API  
|   |   +-- _CONTEXT.md (Phase 3)  
|   +-- worker/       ← Job worker  
|   |   +-- _CONTEXT.md (Phase 2)  
|   +-- dashboard/    ← Frontend React  
|       +-- _CONTEXT.md (Phase 3)  
+-- AGENTS.md         ← Framework condiviso  
+-- CLAUDE.md  
+-- .adlc/            ← Framework condiviso
```

L'agente usa il `_CONTEXT.md` più vicino al file su cui lavora.

Scaffold di un nuovo sottoprogetto:

Listing 15.2: Inizializzazione sottoprogetto in monorepo

```
bash .adlc/tools/init.sh --project-root apps/worker --framework-root .
```

Aggiornamento indice: dopo aggiunte o spostamenti di sottoprogetti, aggiorna `.adlc/projects.json`:

Listing 15.3: Aggiornamento indice progetti

```
bash .adlc/tools/update-projects.sh
```

15.2 Company extension: processi SDLC aziendali

La company extension porta processi SDLC, standard di sicurezza e gate di governance nel contesto dell'agente.

Listing 15.4: Struttura della company extension

```
.adlc/company/  
+-- README.md      ← indice  
+-- PROCESS.md     ← SDLC aziendale  
+-- GOVERNANCE.md  ← approvazioni, compliance  
+-- STANDARDS.md   ← standard ingegneristici  
+-- source/        ← PDF/DOCX originali  
+-- processed/     ← versioni Markdown generate
```

Per convertire documenti aziendali in Markdown leggibile dall'agente:

Listing 15.5: Preprocessing documenti aziendali

```
bash .adlc/tools/preprocess-company-docs.sh
```

NOTA

Le regole della company extension hanno priorità sulle regole del framework. Se l'azienda richiede comportamenti diversi da quelli predefiniti in `.adlc/modules/`, vincono le regole aziendali.

15.3 Codebase legacy: analisi e documentazione

ADLC copre questo flusso con due moduli in sequenza: prima si analizza (`09_CODEBASE_ANALYSIS.md`), poi si documenta (`10_DOCUMENTATION.md`).

Step 0 — Prepara il contesto

Listing 15.6: `_CONTEXT.md` per analisi legacy

```
| Phase | 0-Discovery |
```

Mode	STANDARD	
Active Task	T-DOC-001 Analisi e doc modulo auth	
Active Task Model Level	5	

Step 1 — Analisi con il modulo 09

Listing 15.7: Prompt di analisi codebase

```
Carica .adlc/modules/09_CODEBASE_ANALYSIS.md e analizza src/auth/.

Produci in docs/_analysis/:
1) MAP.md - moduli, dipendenze, entry point
2) RISKS.md - rischi tecnici e debito
3) HOTSPOTS.md - file più critici con motivazione
4) RUN.md - build/test/run

Non inventare: se ti mancano informazioni, elenca ASSUMPTION + TODO.
```

Step 2 — Documentazione con il modulo 10

Quando MAP.md è approvato, genera la documentazione navigabile usando docs/_analysis/ come fonte.

Step 3 — Verifica

Per ogni file generato: verifica che gli endpoint API esistano nel codice, che le entità corrispondano allo schema, che i diagrammi Mermaid renderizzino e che ogni TODO e ASSUMPTION sia giustificato.

ATTENZIONE

Non saltare l'analisi (Step 1). Documentare senza MAP.md produce contenuti inventati. Non usare Mode LITE o RAPID — la documentazione è output ad alto impatto a lungo termine.

Riepilogo

- **Monorepo:** ogni sottoprogetto ha il proprio _CONTEXT.md; l'agente usa quello più vicino al file su cui lavora.
- **Company extension:** .adlc/company/ porta i processi SDLC aziendali nel contesto; priorità sulle regole del framework.
- **Codebase legacy:** il flusso è sempre analisi (09) → documentazione (10). Non invertire l'ordine.

Parte V

In Produzione

16

Troubleshooting, CI/CD e Manutenzione del Framework

Arrivati in produzione, ADLC entra nella sua fase di vita più lunga: manutenzione. Il framework deve rimanere utile mentre il team cresce, gli agenti AI evolvono e il progetto accumula storia.

16.1 Troubleshooting — problemi comuni

“L’agente ignora i vincoli SEC”

Cause: il bootstrap non è avvenuto, `_CONTEXT.md` ha i vincoli nel formato sbagliato, sessione molto lunga con context parzialmente perso.

Soluzione: `@show-constraints`. Se la lista è vuota, chiedi: “Rileggi `_CONTEXT.md` e confermami i vincoli attivi.” Usa `@checkpoint` ogni 3–5 azioni per ri-ancorare il contesto in sessioni lunghe.

“L’agente chiede conferma anche per task LOW”

Causa: Mode STANDARD su progetto maturo dove i task sono routine.

Soluzione: abbassa a LITE in `_CONTEXT.md`.

“L’agente ha modificato file che non doveva”

Causa: il path non è coperto dagli HALT trigger.

Soluzione: aggiungi il path in `.adlc/project/halt-triggers.yaml`.

“Il validator fallisce su un task”

Causa: Model Level o Risk Floor Applied contengono ancora un placeholder.

Soluzione: sostituisci i placeholder con valori reali nel file di task.

“Il debugging non converge”

Soluzione: carica 11_BUGFIX_PLAYBOOK.md. Impone triage strutturato (10 min), root cause analysis, fix minimale, verifica.

“Voglio bypassare un HALT trigger”

Soluzione corretta: non scavalcarlo. Disattivalo consapevolmente per la durata del lavoro intensivo in .adlc/project/halt-triggers.yaml, documenta in PROGRESS.md perché e quando pianifichi di riattivarlo.

16.2 Integrazione CI/CD

Listing 16.1: GitHub Actions per validazione ADLC

```
name: ADLC Validate

on:
  pull_request:
  push:
    branches: [main]

jobs:
  adlc-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Validate ADLC
        run: bash .adlc/tools/validate.sh

      # Con --strict i warning diventano failure
      # - name: Validate ADLC strict
      #   run: bash .adlc/tools/validate.sh --strict

      - name: Sync Copilot check
        run: bash .adlc/tools/sync-copilot.sh
```



```
- name: Smoke tests
  run: bash .adlc/tests/test.sh
```

16.3 Manutenzione del framework nel tempo

ADLC segue semantic versioning (MAJOR.MINOR.PATCH):

Tipo	Esempio	Quando aggiornare
PATCH	3.3.0 → 3.3.1	Subito: solo fix e chiarimenti
MINOR	3.3.x → 3.4.0	Pianificato: nuovi template, tool
MAJOR	3.x.x → 4.0.0	Con attenzione: leggere MIGRATION.md

Tabella 16.1: Policy di aggiornamento del framework

Cosa aggiornare: `.adlc/modules/`, `.adlc/schemas/`, `halt-triggers.yaml`, `manifest.json`, `VERSION`, e i file di startup degli agenti (`AGENTS.md`, `CLAUDE.md`, ecc.).

Non toccare mai durante l'aggiornamento: `_CONTEXT.md`, `PROGRESS.md`, `.adlc/project/`, `.adlc/company/` — questi sono tuoi.

16.4 L'agente in Fase 6 — Ops

In Phase: 6-Ops il comportamento cambia:

- Ogni modifica ha un piano di rollback obbligatorio.
- Le dipendenze si aggiornano solo con test di non-regressione.
- Il runbook è il primo posto dove guardare prima di inventare soluzioni.
- L'incident response segue il workflow del modulo `06_OPS.md`.

Riepilogo

- I problemi comuni hanno cause precise: usa `@show-constraints` per vincoli ignorati, abbassa il Mode per troppe conferme, aggiungi trigger per file toccati indebitamente.
- `validate.sh` in GitHub Actions verifica la presenza e validità del framework ad ogni PR.
- Aggiornamenti: copia solo `.adlc/modules/` e file di startup — mai `_CONTEXT.md` o `.adlc/project/`.
- Phase 6 (Ops): rollback obbligatorio, non-regressione, incident response strutturata.

Parte VI

Appendici



Glossario

Definizioni operative dei termini usati nel libro.

ADLC AI-Driven Software Development Life Cycle. Il framework descritto in questo libro. Non è un'AI, non è un tool da installare — è un insieme di file Markdown e JSON che strutturano il dialogo tra sviluppatore e agente AI.

Agent Bootstrap La sequenza di operazioni che l'agente esegue all'inizio di ogni sessione: legge `_CONTEXT.md`, carica il modulo di fase appropriato, legge le regole di progetto, conferma il contesto.

AI Sizing La stima associata a ogni task ADLC: token di input, token di output, totale, model level, modello raccomandato e motivazione.

ASSUMPTION Uno dei tre confidence tag. Indica che l'agente ha fatto un'ipotesi non supportata da evidenza diretta. Richiede verifica prima di qualsiasi operazione.

Checkpoint Punto fermo fissato con `@checkpoint` ogni 3–5 azioni significative.

Comando conversazionale Istruzione in formato `@keyword` interpretata dall'agente secondo il protocollo ADLC. Non è un comando shell.

Company extension Cartella `.adlc/company/` con processi SDLC aziendali. Ha priorità sul framework.

Confidence tag Classificazione della certezza dell'agente. Tre valori: `FACT` (verificato), `INFERRED` (dedotto), `ASSUMPTION` (ipotesi).

_CONTEXT.md Il file di stato persistente del progetto: fase, task attivo, stack, vincoli SEC e PERF.

CRITICAL Livello di rischio per decisioni mission-critical ambigue. L'agente non esegue nulla. Model Level minimo: 7.

Entry point Il file che un agente AI legge come prima istruzione: `CLAUDE.md` (Claude Code), `AGENTS.md` (Codex), `GEMINI.md`, `OPENCLAW.md`, `.github/copilot-instructions.md` (Copilot).

FACT Uno dei tre confidence tag. L'agente ha verificato l'informazione leggendo direttamente il codice o un file.

HALT L'azione con cui l'agente si ferma prima di modificare un path protetto dagli `HALT` trigger.

HALT trigger Pattern di path che richiedono conferma esplicita prima di qualsiasi modifica.

- HIGH** Livello di rischio per modifiche difficilmente reversibili. Model Level minimo: 5.
- HIGH+** Sottolivello di HIGH per secrets, compliance, dati in produzione. Model Level minimo: 6.
- INFERRED** Uno dei tre confidence tag. L'agente ha dedotto da contesto indiretto senza verifica diretta.
- LOW** Il livello di rischio più basso: modifiche facilmente reversibili. L'agente esegue e notifica.
- MEDIUM** Livello per nuove funzionalità e refactor reversibili. L'agente propone piano e aspetta approvazione. Model Level minimo: 3.
- Mode** Livello di cerimonia: LITE, STANDARD, AUDIT, RAPID, FAST. Gli HALT trigger non vengono mai disattivati.
- Model Level** Scala 1–7 che mappa la complessità del task al modello AI appropriato.
- Modulo** File in `.adlc/modules/NN_*.md`. Read-only. Caricato per fase o su richiesta.
- PERF-XX** Vincolo di performance riusabile. Attivato in `_CONTEXT.md`. Sette predefiniti (PERF-01..PERF-07).
- Phase** Una delle sette fasi ADLC: 0-Discovery, 1-Analysis, 2-Design, 3-Implementation, 4-Verification, 5-Release, 6-Ops.
- PROGRESS.md** Il diario di bordo del progetto. Registra fatto, scoperte e decisioni in ordine cronologico inverso.
- Risk floor** Model Level minimo imposto dal livello di rischio, indipendentemente dalla stima token.
- SEC-XX** Vincolo di sicurezza riusabile. Nove predefiniti (SEC-01..SEC-09).
- Skill** File in `.adlc/modules/skills/SKILL_*.md`. Guida tematica caricata su richiesta.
- Strict mode** Modalità del validator (`validate.sh --strict`) in cui i warning diventano failure. Usato in CI.
- sync-copilot** Tool che verifica l'allineamento tra `AGENTS.md` e `.github/copilot-instructions.md`.
- Validator** Script (`validate.ps1 / validate.sh`) che verifica presenza e validità del framework.

B

Comandi Conversazionali Completi

Riferimento completo di tutti i comandi conversazionali ADLC.

Tabella di riferimento rapido

Comando	Scopo	Bypass sicurezza?
@checkpoint	Fissa stato avanzamento	No
@context-update	Propone aggiornamento context	No
@show-constraints	Mostra vincoli SEC/PERF	No
@security-check	Verifica codice vs SEC	No
@perf-check	Verifica codice vs PERF	No
@load-phase [N]	Carica modulo fase N	No
@set-phase [N]	Propone cambio fase	No
@prompt-list	Lista prompt riusabili	No
@prompt [ID]	Applica template prompt	No
@alternatives	Genera 2–3 opzioni	No
@simplify	Riduce scope/complessità	No
@rollback	Piano di rollback	No
@stop	Ferma immediatamente	No

Tabella B.1: Riferimento rapido comandi ADLC

Regole di tutti i comandi

1. Nessun comando bypassa gli HALT trigger. Neanche @stop li disattiva.
2. Nessun comando modifica file automaticamente — propone sempre, tu applichi.

3. @security-check e @perf-check citano solo i vincoli in _CONTEXT.md.
4. @rollback non esegue azioni distruttive senza conferma esplicita.
5. I comandi non cambiano la classificazione del rischio.

Comandi personalizzati di progetto

In `.adlc/project/instructions.md`, definisci shortcut per workflow ricorrenti:

Listing B.1: Formato comandi personalizzati

```
## Project Commands

@nome-comando
[Descrizione di cosa deve fare l'agente quando
riceve questo comando]
```

Vedi Capitolo 13 per esempi e output attesi di ciascun comando.

C

Template Pronti all'Uso

Template copiabili per i file più usati di ADLC. Sostituisci i valori tra [parentesi] con quelli reali del tuo progetto. I template completi in formato Markdown sono in `.adlc/modules/templates/`.

Template 1 — `_CONTEXT.md` Minimo (spike/POC)

Listing C.1: Context minimale per spike

```
# PROJECT CONTEXT - MINIMAL
> Ultimo aggiornamento: [YYYY-MM-DD]

| Param                | Value                |
|-----|-----|
| Project              | [Nome]               |
| Phase                | [fase]               |
| Mode                 | RAPID                |
| Active Task          | [descrizione task]   |
| Active Task Token Est. | 0/0/0                |
| Active Task Model Level | 3                    |

## Stack
| Runtime | [tecnologia] |
| Framework | [framework] |

## Vincoli Essenziali
| SEC-01 | Input Validation | [approccio] |
| SEC-02 | Auth             | [meccanismo] |

## Note per l'Agente
- Mode RAPID: branch dedicato, non toccare produzione
- Obiettivo time-boxed: [cosa stai esplorando, entro quando]
```

Template 2 — Voce PROGRESS.md

Listing C.2: Voce di sessione per PROGRESS.md

```
## [YYYY-MM-DD] - [ID Task] [Titolo Task]

**Fatto:**
- [cosa è stato implementato]

**Scoperto:**
- [comportamento inaspettato, bug latente trovato]

**Decisione:**
[DECISIONE: descrizione]
Motivazione: [perché questa opzione]

**Prossima sessione:** [cosa fare]

---
```

Template 3 — Task ADLC

Listing C.3: Template di task con AI Sizing

```
# [TASK-ID] - [Titolo del Task]

## AI Sizing
| Token Estimate      | [input] / [output] / [totale] |
| Model Level         | [1-7]                           |
| Risk Floor Applied  | [LOW/MEDIUM/HIGH → min N]     |
| Rationale            | [motivazione]                   |

## Goal
[Cosa deve produrre questo task - 1-3 frasi]

## Acceptance Criteria
- [ ] [criterio verificabile 1]
- [ ] [criterio verificabile 2]

## Vincoli rilevanti
- [SEC-XX o PERF-XX applicabili]
```

Template 4 — Override HALT Trigger di Progetto

Listing C.4: Override di progetto per halt-triggers.yaml

```
version: 1

# Per disattivare un trigger predefinito (temporaneamente):
# disable:
#   - schema

triggers:
  - id: billing
    patterns:
      - "**/billing/**"
      - "**/payments/**"
    reason: Logica di pagamento - impatto finanziario
    risk: HIGH+
```

Template 5 — Skill di Progetto Personalizzata

Listing C.5: Template di skill personalizzata

```
# SKILL - [Nome Skill]

> Carica quando: [tipo di lavoro]

## Identità
Sei [ruolo specializzato]. Priorità: [principio].

## Regole di Dominio
- [regola di business 1]
- [invariante critica]

## Anti-pattern
- NON fare [cosa] - [perché]

## Checklist di Verifica
- [ ] [controllo 1]
```


D

Risorse e Letture Consigliate

Riferimenti per approfondire i temi trattati nel libro.

NOTA

I link a documentazione ufficiale di agenti AI cambiano frequentemente. Verifica che gli URL siano ancora validi prima di seguirli.

Il Framework ADLC

- **Repository ufficiale:** <https://github.com/rollaradio/adlc-framework> — sorgente del framework, CHANGELOG, MIGRATION.md.
- **MANUAL.it.md** (nel repository): manuale tecnico compatto per riferimento rapido.
- **COMMANDS.md:** specifica formale di tutti i comandi conversazionali.

Prompt Engineering e Contesto

- **The Prompt Report** — Schulhoff et al. (2024). Survey sulle tecniche di prompt engineering. Spiega perché il contesto strutturato funziona meglio delle istruzioni inline.
- **Chain-of-Thought Prompting Elicits Reasoning in LLMs** — Wei et al. (2022). Spiega perché “pianificare prima di agire” produce output migliori.
- **Software Design for AI (SDD)** — Fowler, martinowler.com. Prospettiva sull’integrazione degli agenti AI nei processi di sviluppo.

Sicurezza

- **OWASP Top 10 (2021):** <https://owasp.org/www-project-top-ten/>. I vincoli SEC-01, SEC-02, SEC-08, SEC-09 derivano direttamente da OWASP.
- **OWASP ASVS:** <https://owasp.org/www-project-application-security-verification-standard/>. Standard dettagliato per vincoli SEC avanzati.
- **OWASP Logging Cheat Sheet:** riferimento diretto per SEC-08.
- **OWASP SSRF Prevention Cheat Sheet:** riferimento per SEC-09.

Testing e Qualità

- **The Testing Pyramid** — Martin Fowler, <https://martinfowler.com/articles/practical-test-pyramid.html>. Guida la strategia di test del libro.
- **EPAM Testing Pyramid for AI-Assisted Development** — EPAM Systems (2024). Adattamento della piramide per contesti con agenti AI.

AI-Assisted Development

- **GitHub Copilot Impact Study** — GitHub/Accenture (2024). Ricerca quantitativa sull'impatto di Copilot sulla produttività.
- **Salesforce AI Developer Survey** — Salesforce (2024). Evidenzia i problemi di context drift e vincoli dimenticati.
- **Cycode State of AI-Assisted Development** — Cycode (2024). Analisi dei rischi di sicurezza con agenti AI non strutturati.

Architettura

- **Architectural Decision Records** — Michael Nygard: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Il formato ADR usato per decisioni CRITICAL.

Standard di riferimento

- **NIST SP 800-53:** standard di controlli di sicurezza per sistemi informativi. I vincoli SEC di ADLC ne sono parzialmente ispirati.
- **Google SRE Book:** <https://sre.google/sre-book/table-of-contents/>. Capitoli su SLO/SLI (base di PERF-01/02) e incident response (base di 06 OPS.md).

Documentazione agenti AI

- **Claude Code — Anthropic:** <https://docs.anthropic.com/claude-code>
- **GitHub Copilot:** <https://docs.github.com/en/copilot>
- **OpenAI CLI / Codex** (verifica stato attuale — il prodotto è soggetto a variazioni): <https://github.com/openai/openai-codex>

